Data Preprocessing Hackathon Guide

- Data Preprocessing Hackathon — One-Sitting Student Guide
  - Suggested Time Box (≈4.5–5 hours total)
  - Stage 0 — Setup (10 min)
  - Stage 1 — Data Understanding (30 min)
  - Stage 2 — Data Cleaning (75 min)
  - Stage 3 — Data Integration (35 min)
  - Stage 4 — Data Transformation (50 min)
  - Stage 5 — Data Reduction (35 min)
  - Stage 6 — Data Splitting & Leakage Prevention (60 min)
  - Stage 7 — Validation of the Pipeline (30 min)
  - Final Submission Checklist
  - Evaluation Rubric

Data Preprocessing Hackathon — One-Sitting Student Guide

From Data Understanding to Pipeline Validation, using the 17-dataset kit (`preprocessing_datasets.zip`)

Format: Solo or pairs. Complete in one sitting, stage by stage, in order — each stage builds on the output of the previous one, so do not skip ahead. **Tools allowed throughout:** Python (pandas, numpy, matplotlib/seaborn, scikit-learn). All 17 datasets in this hackathon are the tiny **"Dataset A"** files, which are **🛠 manual-first** — write the diagnosis and fix logic yourself before reaching for AI tools. AI may be used only to speed up plotting boilerplate, never to make the diagnostic judgment for you. **Deliverable format:** one working notebook per stage (or one master notebook with 7 clearly marked sections), plus the one-line "diagnosis notes" described in Stage 1 kept running throughout.

Suggested Time Box (≈4.5–5 hours total)

Stage	Topic	Time
0	Setup	10 min
1	Data Understanding	30 min
2	Data Cleaning	75 min
3	Data Integration	35 min
4	Data Transformation	50 min
5	Data Reduction	35 min
6	Data Splitting & Leakage Prevention	60 min
7	Validation of the Pipeline	30 min

Stage 0 — Setup (10 min)

1. Unzip `preprocessing_datasets.zip` into a working folder.
2. Load every file once with `pd.read_csv()` in a loop and print `.shape` for each — confirm all 17 load without error (file `13_customer_records_schema_bug.csv` should still load fine via pandas; its issue is only visible if you naively split on commas — keep that in mind for Stage 2).
3. Create a running markdown cell/log called **"Diagnosis Notes"** — you will add one line to it per dataset throughout the hackathon.

Stage 1 — Data Understanding (30 min)

Concept: Before fixing anything, profile every dataset: shape, dtypes, summary statistics, and value ranges. The goal here is *detection*, not correction — you are not allowed to fix anything in this stage.

Datasets used: all 17 files (quick pass).

Instructions: 1. For each file, run `.info()`, `.describe(include='all')`, and `.isnull().sum()`. 2. For categorical columns, run `.value_counts()`. 3. For numeric columns, plot a quick histogram or boxplot. 4. Write **one line per file** in your Diagnosis Notes: *"This dataset appears to have an X issue, evidenced by Y."* Do this from observation alone — do not look ahead at later stage instructions for the answer.

Checkpoint / Deliverable: A markdown table with 17 rows: `filename | suspected issue | evidence`. This becomes your worksheet for Stages 2–6.

Stage 2 — Data Cleaning (75 min)

Concept: Fix the issues that corrupt individual records or values before any transformation is meaningful — missing values, duplicates, outliers, inconsistent category labels, wrong data types, and noisy text.

2.1 Missing Values

Dataset: `01_missingness_mechanisms.csv` **Task:** Classify each of the 3 missing `income` cells as MCAR, MAR, or MNAR by reasoning about *why* it might be missing (not just that it is). Apply an appropriate fix per mechanism — plain deletion, group-mean imputation, or adding a "missingness flag" feature. **Checkpoint:** Justify in writing why you chose a different fix for at least two of the three cells.

2.2 Duplicate Records

Dataset: `02_duplicate_customers.csv` **Task:** Use `.duplicated()` to catch exact duplicates. Then normalize the `phone` column (strip non-digit characters) and re-check — this should reveal the near-duplicate that exact matching missed. **Checkpoint:** Report before/after row counts.

2.3 Outliers & Noise

Dataset: `03_exam_scores_outliers.csv` **Task:** Compute IQR bounds and Z-scores by hand (or with 2–3 lines of code) for the `score` column. Identify which flagged value is an **impossible** value vs. an **extreme-but-plausible** one, and treat them differently (cap/remove the impossible one; keep and note the plausible one). **Checkpoint:** State your treatment decision for each flagged row and why.

2.4 Inconsistent Categorical Labels

Dataset: `04_city_label_variants.csv` **Task:** Normalize casing and whitespace, then build a manual mapping to merge `Bombay → Mumbai`. Decide whether `New Delhi` and `Delhi` should merge (defend either choice). **Checkpoint:** Report unique category count before and after cleaning.

2.5 Mixed / Incorrect Data Types

Dataset: `06_price_mixed_types.csv` **Task:** Strip currency symbols and thousands-commas from `price`, cast to numeric, and correctly convert `"N/A"` to a true missing value (not the string `"N/A"`). **Checkpoint:** Confirm `df['price'].dtype` is now numeric and `df['price'].mean()` runs without error.

2.6 Text Noise

Dataset: `14_product_reviews_text_noise.csv` **Task:** Build a cleaning pipeline: strip HTML tags → normalize whitespace → lowercase → strip excess punctuation → tokenize. Print vocabulary size after each step. **Checkpoint:** Show the vocabulary size shrinking at each cleaning step in a small table.

End-of-Stage Deliverable: 6 cleaned DataFrames, each with a 2–3 line note on what was fixed and how.

Stage 3 — Data Integration (35 min)

Concept: Reconcile data that comes from different sources or formats into one consistent structure — unify units, and validate/repair schema before combining or trusting any file.

3.1 Inconsistent Units

Dataset: `05_height_mixed_units.csv` **Task:** Plot the 6 `height` values on a strip plot/histogram — visually identify the bimodal split (cm cluster vs. inch cluster). Convert all values to a single unit (cm). **Checkpoint:** Confirm the histogram becomes unimodal after conversion.

3.2 Structural / Schema Errors

Dataset: `13_customer_records_schema_bug.csv` **Task:** First open the raw file in a plain text editor (not pandas) and find rows 2 and 4, where the `address` field contains an unescaped comma. Then load it with `pd.read_csv()` and confirm pandas' quote-handling already parsed it correctly — the lesson here is that the *risk* lives in naive parsing (e.g., manual `.split(',')`), not necessarily in pandas itself. Demonstrate the failure by manually splitting one raw line on commas and showing the column count breaks. **Checkpoint:** Show the column misalignment that a naive parser would produce vs. pandas' correct parse.

End-of-Stage Deliverable: Both datasets in one clean, unit-consistent, schema-validated form, plus a short note explaining the schema risk you demonstrated.

Stage 4 — Data Transformation (50 min)

Concept: Reshape values so they meet the assumptions of downstream models — scale/standardize, encode categories, fix skew, and stabilize time-based data.

4.1 Skewed Distributions

Dataset: `10_income_skewed.csv` **Task:** Compute mean vs. median (note the gap caused by the single

extreme value). Apply a log transform and re-plot the histogram. Separately, discuss/write: is deletion of the outlier a valid fix here, or should you *always* prefer a transform? Justify using this specific dataset's shape. **Checkpoint:** Before/after skewness value (use `.skew()`) and your written justification.

4.2 Cardinality & Encoding

Dataset: `17_customer_id_high_cardinality.csv` **Task:** One-hot encode both `favorite_color` (low cardinality) and `customer_id` (maximal cardinality). Compare resulting column counts and explain in one sentence why encoding `customer_id` is meaningless for modeling. **Checkpoint:** Report column counts for each encoding and your explanation.

4.3 Non-Stationarity (as a transformation problem)

Dataset: `11_monthly_sales_seasonal.csv` **Task:** Plot the series and confirm the seasonal cycle by eye. Apply first-order differencing (`.diff()`) and re-plot — observe the series becoming flatter/more stable around a constant mean. **Checkpoint:** Compare rolling mean/variance before and after differencing.

End-of-Stage Deliverable: 3 transformed datasets with before/after distribution comparisons.

Stage 5 — Data Reduction (35 min)

Concept: Remove redundant or excessive dimensionality before modeling — both because it hurts distance-based methods, and because redundant features destabilize linear models.

5.1 High Dimensionality / Curse of Dimensionality

Dataset: `08_five_points_base_2d.csv` **Task:** Write a short script that pads these 5 points with random noise columns to reach 10D, then 20D. Compute the pairwise distance matrix at each stage and calculate `(max_dist - min_dist) / min_dist`. **Checkpoint:** Show this ratio shrinking as dimensions increase — the numeric proof of "everything becomes equidistant."

5.2 Multicollinearity

Dataset: `09_house_size_multicollinear.csv` **Task:** Compute the correlation between `size_sqft` and `size_sqm` (should be ~1.0). Fit a linear regression on `price_k` using both columns, then again using only one. Compare coefficient stability (note if signs or magnitudes look unstable/unreasonable with both included). **Checkpoint:** State which column you'd drop and why.

End-of-Stage Deliverable: A short written note connecting both exercises to the same underlying idea — "more/redundant features are not automatically better."

Stage 6 — Data Splitting & Leakage Prevention (60 min)

Concept: How you split data — and *when* you fit preprocessing steps relative to that split — determines whether your evaluation metric can be trusted at all. This stage also covers subtler traps: imbalance-aware splitting, biased sampling, and noisy labels discovered only once you start modeling.

6.1 Data Leakage

Dataset: `12_loan_approval_leakage.csv` **Task:** Train a classifier on `approved` twice: once **with** `loan_paid_off` included as a feature, once **without**. Compare accuracy. Write one paragraph

explaining why `loan_paid_off` could not legitimately be known at prediction time. **Checkpoint:** Report both accuracy numbers — the gap should be dramatic and is your proof of leakage.

6.2 Class Imbalance & Stratified Splitting

Dataset: `07_class_imbalance_2d.csv` **Task:** Do a plain random train/test split, then a **stratified** split (`stratify=label`). Check whether the minority class (`label=1`, only 2 points) even appears in both the train and test sets in each case. **Checkpoint:** Report class counts per split for both methods; explain why stratification matters more as the minority class shrinks.

6.3 Sampling Bias

Dataset: `16_survey_daytime_biased_sample.csv` **Task:** Compute the average `response_score` from this sample. Then write (no code needed) what could go wrong if you treated this as representative of a population that is actually ~50% night-shift workers — a bias you cannot detect from the file alone. **Checkpoint:** One paragraph: how would you redesign data collection to avoid this, and how would you detect it if you only had this file and no outside knowledge?

6.4 Label Noise

Dataset: `15_two_cluster_label_noise.csv` **Task:** Plot the 2D points colored by `label`. Visually identify the point that looks mislabeled relative to its neighborhood. Train a simple classifier with and without that point and compare the decision boundary. **Checkpoint:** Screenshot/plot of the boundary shift, plus the row index you flagged.

End-of-Stage Deliverable: A short “Splitting & Trust” writeup covering all four sub-exercises — this is the stage most connected to whether your final reported numbers will hold up.

Stage 7 — Validation of the Pipeline (30 min)

Concept: After cleaning, integrating, transforming, reducing, and splitting correctly, go back and confirm the pipeline actually did what you intended — nothing was leaked, nothing silently reintroduced bias, and distributions look sane.

No new dataset — this stage revisits your own outputs from Stages 2–6.

Instructions: 1. **Re-check distributions:** for at least 2 datasets you transformed (Stage 4), re-plot the “after” histograms and confirm they still look reasonable (no new NaNs, no wrong-signed values, no lost rows you didn’t intend to lose). 2. **Re-confirm no leakage:** revisit `12_loan_approval_leakage.csv` — explicitly state in writing that your final feature set contains no post-outcome information, and that any scaler/imputer you used elsewhere was fit only on training data. 3. **Re-run your Stage 1 diagnosis table:** update it with a new “Fixed? (Y/N) + method used” column for all 17 files. Every row should now say Y. 4. **Final sanity model:** pick any one cleaned + transformed dataset (e.g., `09_house_size_multicollinear.csv` after dropping the redundant column) and fit a simple model end-to-end, confirming the whole pipeline runs without errors from raw CSV to prediction.

Checkpoint / Final Deliverable: The completed 17-row diagnosis-and-fix table, plus a one-paragraph reflection: *“Which single issue, if left unfixed, would have most damaged model performance or trustworthiness, and why?”*

Final Submission Checklist

- [ ] Stage 1 diagnosis table (17 rows, issue + evidence)
- [ ] Stage 2: 6 cleaned datasets with fix notes
- [ ] Stage 3: 2 integrated/schema-validated datasets
- [ ] Stage 4: 3 transformed datasets with before/after distribution comparisons
- [ ] Stage 5: reduction notes for both exercises + coefficient comparison
- [ ] Stage 6: leakage accuracy comparison, stratified split comparison, bias reflection, label-noise plot
- [ ] Stage 7: updated diagnosis table with Fixed column, final sanity model run, closing reflection paragraph

Evaluation Rubric

Component	Weight
Correct diagnosis before fixing (Stage 1 + running notes)	20%
Correct, justified fix per issue (Stages 2–6)	40%
Before/after evidence (numbers, plots, not just claims)	25%
Final pipeline validation & reflection quality (Stage 7)	15%